

VDMS: A System for Efficient Big-*Visual*-Data Access

Luis Remis
Intel Labs
luis.remis@intel.com

Christina R. Strong
Intel Labs
christina.r.strong@intel.com

Vishakha Gupta-Cledat
Intel Labs
vishakha.s.gupta@intel.com

Ragaad Altarawneh
Intel Labs
ragaad.altarawneh@intel.com

ABSTRACT

We demonstrate Visual Data Management System (VDMS), a data management solution that treats visual data such as images, videos, and feature descriptors as first class entities. The goal of VDMS is to enable efficient access of big-*visual*-data to support visual analytics. This is achieved by searching for relevant visual data via metadata stored as a graph, as well as enabling faster access to visual data through new machine-friendly storage formats. VDMS differs from existing large scale photo serving, video streaming, and textual big-data applications due to its primary focus on supporting machine learning and data analytics pipelines that use visual data. We demonstrate how to use VDMS via its user friendly interface as well as how VDMS enables rich and efficient vision analytics through two examples: a medical imaging application and face recognition that is core component for various applications. We also use the medical imaging application to demonstrate the improved performance of VDMS over a comparable set-up.

PVLDB Reference Format:

Luis Remis, Vishakha Gupta-Cledat, Christina R. Strong, Ragaad Altarawneh. VDMS: A System for Efficient Big-*Visual*-Data Access. *PVLDB*, 11 (5): xxxx-yyyy, 2018.
DOI: <https://doi.org/TBD>

1. INTRODUCTION

Visual computing workloads can vary widely, from constructing augmented and virtual reality environments on the fly to performing analytics on streaming video data. This is in part due to the growing ability of machine learning applications to extract information out of the visual data and subsequently use that information. The insights this information can provide depend on the application: a retail vendor might be interested in the amount of time shoppers spend in front of a specific product, while a medical expert might want to see the effect of a specific treatment on the size of a tumor.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Articles from this volume were invited to present their results at The 44th International Conference on Very Large Data Bases, August 2018, Rio de Janeiro, Brazil.

Proceedings of the VLDB Endowment, Vol. 11, No. 5

Copyright 2018 VLDB Endowment 2150-8097/18/1.

DOI: <https://doi.org/TBD>

Despite this rich and varied usage environment, there has been very little research on the management of visual data. Most current storage solutions are an ad-hoc collection of tools combined with custom scripts to tie them together, unique not only to a specific discipline but often to individual researchers. Not only do these solutions make replicating experiments difficult, they do not scale well to real-world applications. Addressing the storage and retrieval of visual data necessitates a complete overhaul of the storage architecture, using emerging breakthroughs in heterogeneous memory and storage.

We present the Visual Data Management System (VDMS) [5], designed to enable efficient access of visual data. Since visual data often contains rich metadata (such as objects, locations, and time), VDMS stores this information in a high performance graph database. Using this metadata, VDMS can quickly identify which data is relevant to a given query. Additionally, VDMS uses a custom library to store and retrieve visual data, which provides an interface for machine friendly formats as well as traditional formats. These new formats are designed to support applications that are often interested in specific areas of images or videos, particularly when the individual object is large.

Combined with a request server that seamlessly handles client queries, VDMS manages all aspects of visual data in a user friendly manner. While comparisons can be made with the individual components of VDMS, there is no one system today that combines them and offers an API that allows for metadata queries as well as data processing through simple, unified commands. In order to compare VDMS [5] to a reasonable state of the art, it was therefore necessary to build our own ad-hoc system from MemSQL [9] for metadata, Apache HTTP Server to serve visual objects, and OpenCV [2] to perform transformations on them when needed.

While there are a number of big-data frameworks [12, 13], systems that can be used to store metadata [9, 8], and systems that manipulate a specific category of visual data [3, 1], VDMS can be distinguished from them by the following characteristics:

- Focus on visual data for machine learning: VDMS specifically targets visual data (images, videos, and feature vectors) for use cases that require manipulation of information captured from these objects. There is an element of scale, both in terms of individual sizes and the number of these objects, that makes it necessary to organize the data and exploit any available hardware features that lend themselves to improving

visual data access performance. VDMS is defined to address these challenges as discussed in Section 2.

- **Ease-of-use:** By defining a common API that allows applications to combine their complex metadata searches with operations on resulting visual data, VDMS goes beyond the traditional SQL or OpenCV level interfaces that do one or the other. Given our focus on enabling machine learning, we also provide a client API in Python.
- **Performance:** We demonstrate how a unified system such as VDMS can outperform an ad-hoc system constructed with well-known discrete components. Because of the capabilities we have built into VDMS, it handles complex queries significantly better than the ad-hoc system without compromising the performance of simple queries.

We demonstrate the user friendly API by constructing queries related to a set of medical images. These queries will also help illustrate the capabilities of VDMS, as well as show how it enables the offloading of some common visual pre-processing steps to where the data resides. In addition, we show how VDMS allows applications to focus on only relevant data through the efficient metadata search. We also demonstrate the unique aspects of our custom library, including a novel image format and feature vector management capabilities.

2. DESIGN AND IMPLEMENTATION

VDMS is a data management system for efficient access of big-*visual*-data that consists of a client and a server. The VDMS Client provides both Python and C++ bindings to enable communication between applications and the VDMS Server. The VDMS Server handles concurrent client requests and coordinates request execution across the metadata and data components in order to return a unified response.

Figure 1 shows the individual components of the VDMS Server. The metadata component is the Persistent Memory Graph Database (PMGD) described next. The Request Server coordinates communications from PMGD with accessing data through our custom library, the Visual Compute Library (VCL), described briefly in Section 2.2. The VCL enables machine friendly enhancements to visual data for faster access. These components are all available as open source tools¹.

2.1 Persistent Memory Graph Database

Recent developments in persistent memory technologies like 3D XPoint [6] promise storage elements providing nearly the speed of DRAM and the durability of block-oriented storage. To provide an efficient storage solution addressing the increasing popularity of connected data and applications that benefit from graph like processing, we have designed and implemented an in-persistent-memory graph database, PMGD, optimized to run on a platform equipped with persistent memory.

PMGD provides a property graph model of data storage with the traditional atomicity, consistency, isolation and durability properties expected from databases. The graph model makes it very suitable for the data model and access patterns shown by visual metadata. Due to its capability

¹<https://github.com/IntelLabs/{vdms, pmgd, vcl}>

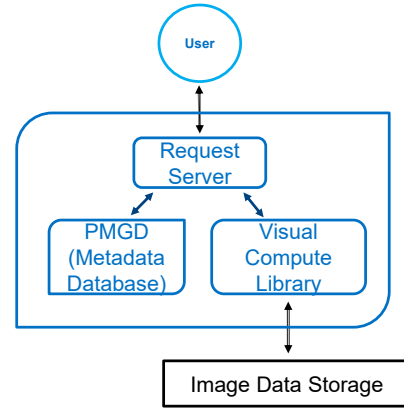


Figure 1: Visual Data Management System Server

to exploit new memory technology, we can guarantee high performance, unlike with traditional graph databases that suffer when traversing pointers for connected queries with data laid out on slower disks. With its support for extending schema very easily, we can support new developments in machine learning that can lead to enhancements to existing metadata over time.

2.2 Visual Compute Library

The Visual Compute Library was designed to provide an interface through which users can interact with visual data. For traditional formats, the interface is an abstraction layer over existing software. However, it also provides a way to use novel formats that are better suited for visual analytics. Currently, the VCL provides support for images and feature vectors, with limited support for videos.

The VCL Image class enables the use of a novel, array-based lossless image format called TDB. TDB is built on the array data manager TileDB [11] and is well suited for images that are used in visual analytics. Feature vector support is provided by the VCL Descriptors class, which contains both storage and search functionality.

2.3 API

One of our goals with VDMS was to define an API that is easy to use. We define a set of JSON-based API calls that allow an application to interact with VDMS and exploit its strengths. We provide C++ and Python client libraries for linking with any application to enable VDMS communication. Our API explicitly predefines certain visual primitives associated with images, videos, and feature vectors. In addition, while we use a graph database to store our metadata, the API is not graph specific.

A user application can use the VDMS API by defining metadata conforming to the query protocol we have defined and passing along blobs. The client side VDMS library provides a simple query function that accepts a JSON string with commands and an array or vector of blobs. Internally, the library wraps the query string and blob in a protobuf and sends it to VDMS. It also receives a similarly wrapped response from VDMS and returns it to the client. The responses require JSON parsing on the client side for the metadata string that indicates how to interpret blobs.

3. DEMONSTRATION OVERVIEW

The demonstration consists of two parts: (1) We walk through the setup of VDMS and its configuration steps and (2) we show the ease-of-use of VDMS through the API for storing and querying visual data as well as its performance benefits. The target audience of the demo includes anyone who develops or deploys machine learning or data analytics applications that deal with large scale visual data. We start by first describing two distinct application use cases where using VDMS is beneficial.

3.1 Use Cases

We have been working on several application use cases to improve our interface and enrich it with functionalities that are typically needed in our target applications.

Medical imaging: Standards for medical images, such as DICOM and NIfTI, were developed to assist in transmitting medical images along with their associated metadata. This metadata, consisting of patient information and often including treatments, can be used together with the images themselves in machine learning applications to gather insights. This makes medical imaging an excellent use case to demonstrate the metadata query and data pre-processing capabilities offered by VDMS. It also helps us explain how queries can be made increasingly complex to give more detailed information, finding the most relevant subset of patients for focused analyses.

Face recognition: Feature vectors are intermediate results of various machine learning or computer vision algorithms when run on visual data. These vectors can be labeled and classified to build search indexes, and there are many in-memory libraries that are designed for this task [10, 7]. VDMS is unique in recognizing feature vectors as first class entities and provides an interface to manage and search stored feature vectors together with the rest of the metadata and visual data. This provides added value, as applications no longer need to rely on a combination of tools and complex integrations between systems and languages. Using feature vectors to recognize faces is a common use case across various applications such as digital surveillance, retail tracking, and photo libraries. We use this common use case of feature vectors as our second example to show how the VDMS API simplifies the complexity of maintaining indexes and searching feature vectors.

3.2 Environment

Our VDMS server runs in a Linux environment and is written in C++. However, we provide client libraries for Python as well as C++ applications. The testbed used for this demonstration is a Macbook Pro as a client used to run Python code through a Jupyter Notebook, and a remote Server with an Intel Xeon E5-2699 v3 CPU running Ubuntu 16.04, Python 2.7, and gcc 5.4.

3.3 Demonstration Scenarios

In this demo, we will introduce our system and show results from the two distinct use cases that we have built on top of our visual storage system. We will first install VDMS on a server from our open source code and show the ease of setting up both server and client(s). We will then run the examples described below to show how queries can be written and executed against our system. At the end of the demo, we will also talk about the ongoing and future work

as well as answer any further questions about the API, our system, or the use cases we have worked with.

We will start by talking about how a schema needs to be defined for an application in order to understand the entities in the system and their relationships to each other. While there is no need to enter the schema in the system, as needed for relational databases where the tables and their columns need to be defined before starting, a schema in our case helps with keeping the API calls consistent between input and queries. We will also talk about how to index properties that will be searched often in order to give a complete usage scenario to the attendees.

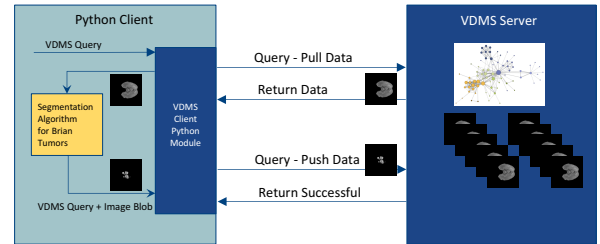


Figure 2: Segmentation pipeline to find brain tumors in existing patient brain scans

We will then demonstrate the medical imaging use case that processes brain scans using The Cancer Image Archive dataset [4] for both metadata and image information. We will present our findings after performing three queries based on both metadata and visual data (brain scans). Finally, we will run a pipeline that uses VDMS to feed brain images to a machine learning component that runs a segmentation algorithm, the results of which are pushed back to storage for future use, as shown in Figure 2. We will use this pipeline to show how applications can interact with VDMS using our API, with a specific focus on the Image API provided by VDMS.

Some of the queries that we will address include:

- Retrieve a single image from a brain scan, search by its unique name, and apply pre-processing operations: Simple case where a single command on the API is used.
- Retrieve a complete brain scan (155 images) from a particular patient, and apply pre-processing operations: Involves performing a neighbor search on the metadata graph, plus accessing multiple images.
- Retrieve all brain scans corresponding to people over 75 who had a chemotherapy using the drug Temodar: Involves performing a more complex metadata graph traversal, plus accessing images corresponding to multiple brain scans.

The second part of our demo will showcase an application that relies on feature vectors to match new faces to faces that have been seen before. We will cover step by step instructions of how an application inserts the images, metadata associated with the images, and feature vectors corresponding to the faces of the people present in the images. We will also demonstrate how an index is created and how associations between feature vectors and other data is specified using our simple yet powerful API. We will then perform a classification task on new images with unknown faces.

Some of the queries that we will address include:

- Insert a new image with the associated metadata and information of all the people present in the image: Involves performing insertion queries as well as the necessary commands to express the relationships.
- Create an index for feature vectors and insert the feature vectors associated with the faces in the images: Involves using the API commands to manage feature vector indexes, querying previously inserted elements (images), and expressing relationships between them and the newly inserted elements (the new feature vectors).
- Classify a new face found in a new image and retrieve all the images in which that face is present: Involves using API commands specifically designed to facilitate feature vector matching, as well as a metadata graph traversal using the matched feature vectors as a starting point.

3.4 User Interface

```
import vdms

db = vdms.VDMS()
db.connect("hsw3.jf.intel.com")

#Get information about all the patients
query = """
{
  "FindEntity": {
    "class": "patient",
    "results": {
      "list": [ "bcr_patient_barco", "gender", "age_at_initial",
    ],
    "sort": "age_at_initial"
    },
    "constraints": { "age_at_initial": [">=", 85]}
  }
}
"""

response, images = db.query(query)
vdms.aux_print_json(str(response))

{
  {
    "FindEntity": {
      "status": 0,
      "entities": [
        {
          "bcr_patient_barco": "TCGA-76-4928-0",
          "gender": "FEMALE",
          "age_at_initial": 85
        },
        {
          "bcr_patient_barco": "TCGA-12-1600-0",
          "gender": "MALE",
          "age_at_initial": 86
        }
      ]
    },
    "returned": 2
  }
}
```

Figure 3: Snapshot of a Python notebook with a simple query client

As mentioned before, VDMS provides Python and C++ client libraries with mainly one function: a query that passes metadata as a JSON string and, optionally, data as a vector of blobs. The VDMS client libraries manages the rest. Figure 3 shows a snapshot from a Jupyter notebook with a query from the medical imaging use case. We have paid particular attention to hide the complexities of our internal implementation and uplevel the API to a JSON-based API which is very popular across various application domains.

3.5 Performance

Figure 4 shows a sample from the performance evaluation we have performed comparing VDMS to the ad hoc baseline we listed earlier, using the medical imaging queries described above. We will demonstrate more aspects of this

performance during the demo but the figure indicates how VDMS is able to perform very well for complex queries without hurting performance of simple queries. It benefits significantly from co-locating some pre-processing on images before data is transferred to an application.

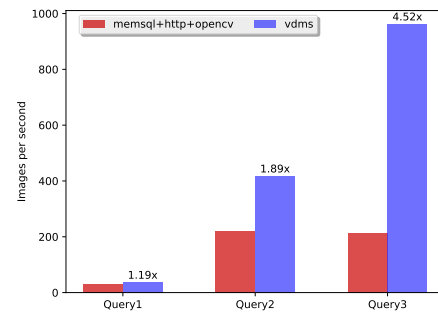


Figure 4: Performance evaluated using the medical imaging queries

4. REFERENCES

- [1] P. Baumann, A. Dehmel, P. Furtado, R. Ritsch, and N. Widmann. The multidimensional database system rasdaman. In *Proc. of the 1998 ACM SIGMOD*, SIGMOD '98, pages 575–577. ACM, 1998.
- [2] G. Bradski and A. Kaehler. *Learning OpenCV: Computer Vision in C++ with the OpenCV Library*. O'Reilly Media, Inc., 2nd edition, 2013.
- [3] P. G. Brown. Overview of SciDB: Large scale array storage, processing and analysis. In *Proc. of the 2010 ACM SIGMOD*, pages 963–968. ACM, 2010.
- [4] Frederick Nat. Lab for Cancer Research. The cancer image archive. <http://www.cancerimagingarchive.net/>.
- [5] V. Gupta-Cledat, L. Remis, and C. R. Strong. Addressing the dark side of vision research: Storage. In *9th USENIX Workshop on HotStorage*. USENIX Association, 2017.
- [6] IntelPR. Intel and micron produce breakthrough memory technology, 2015.
- [7] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with gpus. *CoRR*, abs/1702.08734, 2017.
- [8] A. Lamb, M. Fuller, R. Varadarajan, N. Tran, B. Vandiver, L. Doshi, and C. Bear. The vertica analytic database: C-store 7 years later. *Proc. of the VLDB Endowment*, 5(12):1790–1801, 2012.
- [9] MemSQL Inc. The real-time data warehouse you can run anywhere. <https://www.memsql.com/>.
- [10] M. Muja and D. G. Lowe. Scalable nearest neighbor algorithms for high dimensional data. *IEEE Trans. on Pattern Analysis and Machine Intelligence*, 36, 2014.
- [11] S. Papadopoulos, K. Datta, S. Madden, and T. Mattson. The tiledb array data storage manager. *Proc. VLDB Endowment*, 10(4):349–360, Nov. 2016.
- [12] The Apache Software Foundation. Apache spark: Lightning-fast cluster computing. <https://spark.apache.org/>.
- [13] The Apache Software Foundation. What is apache hadoop? <https://hadoop.apache.org/>.